Context Management

How `mini-coding-agent-js` spends its context window — and how production agent systems decide what to keep, drop, compress, retrieve and isolate.

> Companion to `explainer-harness-js.md` and `README.md`. Read `src/agent.js` first: the whole agent is the `messages` array plus a `for` loop. This document is about the array.

CONTENTS

What is in this explainer

The short version 3

1. What a context window actually is 4

2. Why agents burn context faster than chatbots 6

3. The context assembly line 7

4. The eight controls 8

5. A worked budget 12

6. What is deliberately missing 13

7. The four moves 14

8. Technique catalogue 15

9. Putting it together: a reference architecture 25

10. Complexity and trade-offs 26

11. Failure modes checklist 28

12. How to evaluate context management 29

13. Exercises for this repository 30

14. Glossary 31

15. Further reading 32

16. The one thing to remember 33

Page numbers refer to the folio at the foot of each page.

The short version

`messages` is the agent's entire working state. This app keeps it small in one simple way: **it stops any one message from getting too big, and it limits how many messages a task can add.** It does not count tokens, summarise old turns or trim the conversation during a run.

WHERE	CONTROL	WHAT IT DOES IN PRACTICE
Before the run	Memory digest	At most 12 entries / 1,200 characters are added to the system prompt; the rest is left in the file and the prompt says so
Before the run	Session history	Only the last 24 messages are carried into a new task, cut at a `user` turn
During the run	`MAX_STEPS = 12`	The loop cannot add unlimited assistant/tool turns
During the run	Tool-output caps	`read_file` 200 lines by default (max 2,000), `list_files` 200 lines, `run_command` 6,000 chars per stream
During the run	Memory tool caps	300 chars per entry, 2,000 per write, 60 entries in the file
Every reply	`max_tokens: 2048`	The model cannot write a whole file into one answer; a truncated reply is not dispatched
Every reply	`sanitizeAssistant`	The model's private reasoning is stripped instead of being re-sent every step
Extension point	Hooks	`PreModel` can measure the request; `PostToolUse` can shrink or redact a tool result

What it does not do:

- **No token counting** — it cannot know in advance that a request will not fit.
- **No trimming or compaction during a run** — `messages` only grows until the loop ends.
- **No summarisation, retrieval or sub-agents** — the memory digest is bounded, but the conversation itself is only cut between tasks.

That is enough for a 12-step demo. Long runs can still overflow, especially because `read_file` is limited by **lines, not bytes**: a few 2,000-line reads can be enormous.

The wider lesson: production systems add four moves — **write** state outside the window, **select** only what is relevant, **compress** what remains, and **isolate** work in separate windows. The rest of this document explains those, their trade-offs, and how you would add them here.

Part I — The constraint

1. What a context window actually is

A language model does not remember. Every request carries the whole conversation: system prompt, tool schemas, user turns, assistant turns, tool results. The provider tokenises all of it, runs it through the model, and returns the next message. Nothing is retained server-side between calls.

That gives three facts that shape every design decision:

1. **Every token is re-sent every step.** A conversation that grows by 2,000 tokens per step does not cost `steps × 2,000` input tokens; it costs roughly `steps² / 2 × 2,000`, because step *n* sends everything from steps *1…n-1* again.
2. **Every token competes for attention.** A larger window is not a larger working memory in the human sense. Models attend unevenly; information in the middle of a long context is used less reliably than information at the start or end (the "lost in the middle" effect), and accuracy on many tasks degrades as input length grows even when the needle is present ("context rot").
3. **Every token has a price and a latency cost.** Prefill time and input cost scale with the prompt. Cached tokens are cheaper and faster, but only if the prefix stays stable.

There are really **four budgets**, and they are not the same number:

BUDGET	WHAT IT LIMITS	WHO SETS IT
Capacity	The model's maximum input + output tokens (e.g. 32k, 128k, 200k, 1M)	Model/provider
Quality	The length at which this model still reasons reliably on this task	Empirical; often well below capacity
Cost	What you are willing to spend per task and per session	Product/business
Latency	How long a user will wait for the next token	UX

A 1M-token window does not give you a 1M-token quality budget. It gives you room to make more expensive mistakes.

Inputs versus outputs

Two token counts matter separately:

- **Input tokens** — everything you send. Billed per request, often cheaper when cached.
- **Output tokens** — what the model generates. Usually more expensive per token, and capped by `max_tokens`.

Output is not free context either: every assistant message you append becomes input on the next step. A model that writes 2,000 tokens of reasoning per step doubles the cost of the run even before tool results arrive.

The token-counting problem

Characters are not tokens. English prose averages roughly **4 characters per token**; code, JSON, paths and punctuation are often denser. Tool schemas and chat templates add tokens you never see in the message text. Images have their own accounting. Approximations are fine for a demo, dangerous for a hard budget.

Production systems use one of:

- a provider token-count endpoint (authoritative for that model),
- a local tokenizer matching the model family (fast, approximate),
- a conservative `chars / 3` or `chars / 4` heuristic plus a safety margin.

The rule of thumb: **never budget to 100% of the window.** Reserve output tokens, template overhead and 10–20% headroom.

---

2. Why agents burn context faster than chatbots

A chat turn adds one user message and one assistant message. An agent turn adds:

```
assistant message (content + tool_calls)
  + tool result 1
  + tool result 2
  + ...
  + the next assistant message
```

And every step re-sends the tool schemas, the system prompt and all previous observations.

Typical agents see context grow from five causes:

CAUSE	EXAMPLE	TYPICAL SIZE
Tool schemas	8 tools described in JSON Schema	~1k tokens here
System prompt	rules, policies, memory block	~0.5–2k tokens
File contents	`read_file`, search results, diffs	100s–10,000s of tokens
Command output	test logs, build logs, stack traces	100s–10,000s of tokens
Model scratchpad	reasoning text, retries, apologies	100s of tokens per step

Code agents are especially exposed because the natural unit of work — a file — is much larger than the natural unit of reasoning — a function. A single `read_file` on a 2,000-line file can consume more context than fifty conversational turns.

Part II — How this app manages context

3. The context assembly line

Every task starts by building a `messages` array in `src/agent.js`:

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  ...history.filter((m) => m.role !== 'system'),
  { role: 'user', content: task },
];
```

The pieces, and how each behaves as the run proceeds:

PIECE	SOURCE	GROWS DURING A RUN?	BOUND
System rules	`prompt.js`	No	Fixed prose, ~1,250 chars
Memory digest	`memory.js`	No (built once per task)	12 entries / 1,200 chars + header
Tool schemas	`tools.js`	No (sent every request)	8 tools, ~4,244 chars of JSON
Prior session turns	`cli.js` → `history`	No within a task	24 messages, cut at a `user` turn
Task	caller	No	User-supplied
Assistant turns	model	Yes, every step	`max_tokens: 2048` per reply
Tool results	`runTool`	Yes, every step	Per-tool caps (below)

The system prompt is deliberately **rebuilt**, not carried in history. That way a memory written by task one is already in front of the model in task two, and a stale system prompt can never drift from the current rules or memory file.

```
// cli.js — carried across tasks, not within a run
history = trimHistory(result.messages);
```

Within a single `runTask`, `messages` is **never trimmed**. The only thing that stops growth is `maxSteps` and the per-tool caps. That is the central fact to understand about this app's context handling.

4. The eight controls

4.1 Bound the loop: `MAX_STEPS = 12`

`config.js` defaults `MAX_STEPS` to 12. The loop stops after 12 model calls, prints an error, fires `StepLimit` and `SessionEnd`, and returns `{ ok: false, reason: 'max_steps' }`.

This is context management by **budgeting the number of growth events**. It does not cap the size of each event, but it caps how many events there can be. Twelve steps of ordinary work fit comfortably in most modern windows; twelve steps of maximal tool output do not.

Why it is a good default: it is predictable, cheap, easy to reason about, and it turns an unbounded failure into a visible one. Why it is not enough: a single 2,000-line `read_file` can use more context than all twelve steps combined.

4.2 Bound observations at the source

The cheapest context management is not generating the tokens in the first place. The tools do this themselves:

TOOL	BOUND	WHERE
`list_files`	200 lines, depth 1–6 (default 3)	`tools.js`
`read_file`	default 200 lines, max 2,000 lines per call	`tools.js`
`run_command`	stdout clipped to 6,000 chars, stderr clipped to 6,000 chars	`clip()` / `MAX_OUTPUT`
`remember`	2,000 chars per call, 300 chars per stored entry	`memory.js`
`recall`	default 20 entries, each ≤300 chars	`tools.js` / `memory.js`
`edit_file`	returns one summary line, not the file	`tools.js`
`create_file` / `write_file`	returns a byte count, not the content	`tools.js`

`read_file` is the most interesting. It is **line-bounded, not character-bounded**:

```
const limit = Math.min(Math.max(Number(args.limit ?? 200), 1), 2000);
const slice = all.slice(offset - 1, offset - 1 + limit);
```

A 2,000-line file of short lines is fine. A 2,000-line file of generated minified code, SQL dumps or long stack traces can be enormous. The model is expected to use `offset`/`limit` intelligently; the harness only sets the ceiling. Production systems usually add a byte/token cap as well, or return a pointer to the full output.

`run_command` clips each stream separately, so the worst case is roughly 12,000 chars (plus labels) per call — still ~3,000 tokens, but bounded and predictable.

4.3 Bound injected memory: eager digest plus lazy recall

Long-term memory is a file, `src/memory.md`. The important context decision is how much of it enters the prompt:

```
// memory.js — the eager path, called once per task by buildSystemPrompt()
digest({ entries = 12, chars = 1200 } = {}) {
  const all = this.entries();
  const newest = all.slice(-entries).reverse();
  // add lines until the character budget is hit
  ...
  const hidden = all.length - lines.length;
  const tail = hidden > 0
    ? `(${hidden} older memories not shown - search ${this.label} with the recall tool)`
    : '';
}
```

Two properties matter:

- **It is bounded twice** — by entry count and by characters. The file can grow to `maxEntries = 60`, but the prompt block cannot grow with it.
- **It says what it left out.** The final line tells the model that older notes exist and how to get them. That turns a silent truncation into an invitation to retrieve.

The lazy path is the `recall` tool: a substring search over the whole file, default 20 hits, only when the model asks. This is the same **eager vs lazy** trade-off production systems make with memory files, retrieval indexes and databases.

The file itself is bounded on write:

- duplicate text refreshes the date instead of appending,
- `maxEntryChars = 300` prevents one essay from dominating,
- `rememberChars = 2000` caps a single call,
- `maxEntries = 60` drops the oldest entries.

As the README puts it: an unbounded memory file is "a context leak with a friendly name."

4.4 Bound the model's output

`llm.js` sends `max_tokens: 2048` on every request:

```
body: JSON.stringify({ model, messages, tools, max_tokens: 2048 }),
```

This does two jobs: it keeps each reply cheap, and it prevents the model from dumping a whole file into a single assistant message. The cost is truncation, which the loop handles explicitly:

```
if (choice.finish_reason === 'length' && !message.tool_calls?.length) {
  messages.push({ role: 'assistant', content: message.content ?? '' });
  messages.push({ role: 'user', content: 'You were cut off by the length limit. Continue with smaller tool calls.' });
  continue;
}
```

The guard exists because a truncated reply often ends mid-JSON. Dispatching `{"path": "hello.js", "cont` would produce a parse error for a tool call the model never finished, and the model would waste a step apologising for a bug it did not have.

Note the subtlety: the truncation guard **adds** context (the assistant's partial content plus a corrective user message) to prevent a worse context outcome. Sometimes spending tokens is the cheapest fix.

4.5 Strip the model's private scratchpad

Providers may return extra fields on assistant messages — `reasoning`, `reasoning_content`, refusal blocks, annotations. `sanitizeAssistant` keeps only what the agent needs:

```
export function sanitizeAssistant(message) {
  const out = { role: 'assistant', content: message.content ?? '' };
  if (Array.isArray(message.tool_calls) && message.tool_calls.length > 0) {
    out.tool_calls = message.tool_calls;
    if (message.content == null) delete out.content;
  }
  return out;
}
```

Echoing reasoning traces back on every later step re-sends the model's private thinking: tokens you pay for, and a context that fills with stale reasoning that may contradict the current plan. Stripping them is a **lossless-for-the-task** compression — the decision and the tool calls remain.

Caveat: some providers require certain blocks to be preserved (for example, Anthropic extended thinking blocks during a tool-use turn). Provider-specific rules beat generic hygiene. In this app the target is an OpenRouter model, and the sanitizer is deliberately conservative about what it drops.

4.6 Trim session history at a safe boundary

`MAX_HISTORY = 24` in `cli.js`. Between tasks, the session carries the tail of the previous conversation:

```
export function trimHistory(messages, max = MAX_HISTORY) {
  const kept = (messages ?? []).filter((m) => m.role !== 'system');
  let start = Math.max(0, kept.length - Math.max(0, max));
  // Never start on a tool result or an unanswered tool_calls: providers reject it.
  while (start < kept.length && kept[start].role !== 'user') start += 1;
  return start >= kept.length ? [] : kept.slice(start);
}
```

Three design decisions are visible:

1. **The system message is excluded**, because `runTask` rebuilds it (with a fresh memory digest).
2. **The cut is positional**, not semantic. It keeps the newest 24 messages, then walks forward until it lands on a `user` turn.
3. **The boundary is structural.** A provider rejects a history that starts with a `tool` result (no preceding `tool_calls`) or ends with an unanswered `tool_calls`. Cutting on a `user` turn avoids both.

The cost: the cut discards the **beginning** of the conversation, which is usually where the original goal, constraints and early discoveries live. A follow-up like "now add tests" may still work because the recent turns contain the code; a follow-up like "why did we choose the CSV format?" may not, because the decision was trimmed away.

This is **sliding-window truncation**, the simplest possible context policy. It is not a summary, and it is not token-aware: 24 messages can be 2,000 tokens or 100,000, depending on what is in them.

4.7 Rebuild the system prompt every task

Because `history` excludes the system message and `runTask` calls `buildSystemPrompt` fresh, memory writes from the previous task are visible immediately. This is a small but important context-management pattern:

- **Stable facts** live in the system prompt (rules, memory digest).
- **Mutable conversation** lives in history.
- **The boundary between them is explicit**, so you can change the stable part without rewriting the conversation.

It also means the prefix changes between tasks, which matters if you later add prompt caching: a changed system prompt invalidates the cache for everything after it.

4.8 Hooks as context-management seams

The loop exposes nine lifecycle events. Five are directly useful for context work:

HOOK	PAYLOAD	WHAT YOU CAN DO TODAY
`SessionStart`	task, workspace, config	Load project rules, open a transcript, check a budget
`PreModel`	step, maxSteps, messages, tools	**Measure** the request; log token estimates and the largest messages
`PostModel`	message, finishReason	Count output tokens, spot loops
`PostToolUse`	name, result, ok	**Replace** an observation: cap it, redact it, or turn it into a pointer
`StepLimit` / `SessionEnd`	steps, messages	Persist a transcript, write a summary, emit cost telemetry

`PreModel` currently **observes only** — its return value cannot replace `messages`. `PostToolUse` is the one hook that can change what the model reads back, which makes it the natural place for clipping, redaction and "full output saved to a file" pointers.

A production version of this loop would let `PreModel` return a new `messages` array. Then compaction, observation masking and budget enforcement become hooks rather than loop edits. That is the extension-point design the hooks chapter argues for, applied to context.

---

5. A worked budget

Approximate sizes for this repository, using ~4 chars per token:

COMPONENT	CHARS	~TOKENS
System rules	1,251	310
Memory digest (this repo, 11 entries)	1,398	350
Tool schemas (8 tools, JSON)	4,244	1,060
Task	~100	25
Base prompt	**~7,000**	**~1,750**

Now add a step. Suppose each step produces:

- 400 tokens of assistant content,
- one `read_file` of 200 lines (~8,000 chars ≈ 2,000 tokens),
- a few tool-call argument tokens.

That is roughly **2,500 tokens added per step**. After 12 steps the array is about `1,750 + 12 × 2,500 ≈ 32,000` tokens — comfortably inside a 128k window.

But the **cumulative input** is much larger, because each step re-sends the whole array:

```
step 1:  1,750 + 2,500      =  4,250
step 2:  1,750 + 5,000      =  6,750
...
step 12: 1,750 + 30,000     = 31,750
                        sum ≈ 216,000 input tokens
```

A 12-step run with modest observations can bill **over 200k input tokens** while the final context is only ~32k. That is the quadratic cost of agents. Prompt caching, compaction and observation masking all attack this number, not just the final window size.

Now the worst case:

- `read_file` at `limit: 2000` on a file with 60-char lines → ~120,000 chars ≈ **30,000 tokens in one call**.
- `run_command` with 6,000 chars of stdout and 6,000 of stderr → ~**3,000 tokens**.
- `recall` at 20 entries × 300 chars → ~**1,500 tokens**.

Two or three maximal reads can exhaust a 128k window on their own. The app has no check that says "this request will not fit"; it relies on the model choosing sensible arguments and on `MAX_STEPS` limiting the damage. A provider will simply reject an over-limit request, and the run ends with `request failed`.

This is the honest summary: **the app bounds the number of steps and the size of most observations, but it does not bound the total.** It is a demo-sized policy, not a production one.

---

6. What is deliberately missing

The README is explicit:

> No context management: a session carries only the last 24 messages, and the cut is positional … Long runs still grow until they hit that cap, and nothing is compacted or retrieved — the memory digest is bounded, the conversation is only truncated.

Missing pieces, and what each would buy:

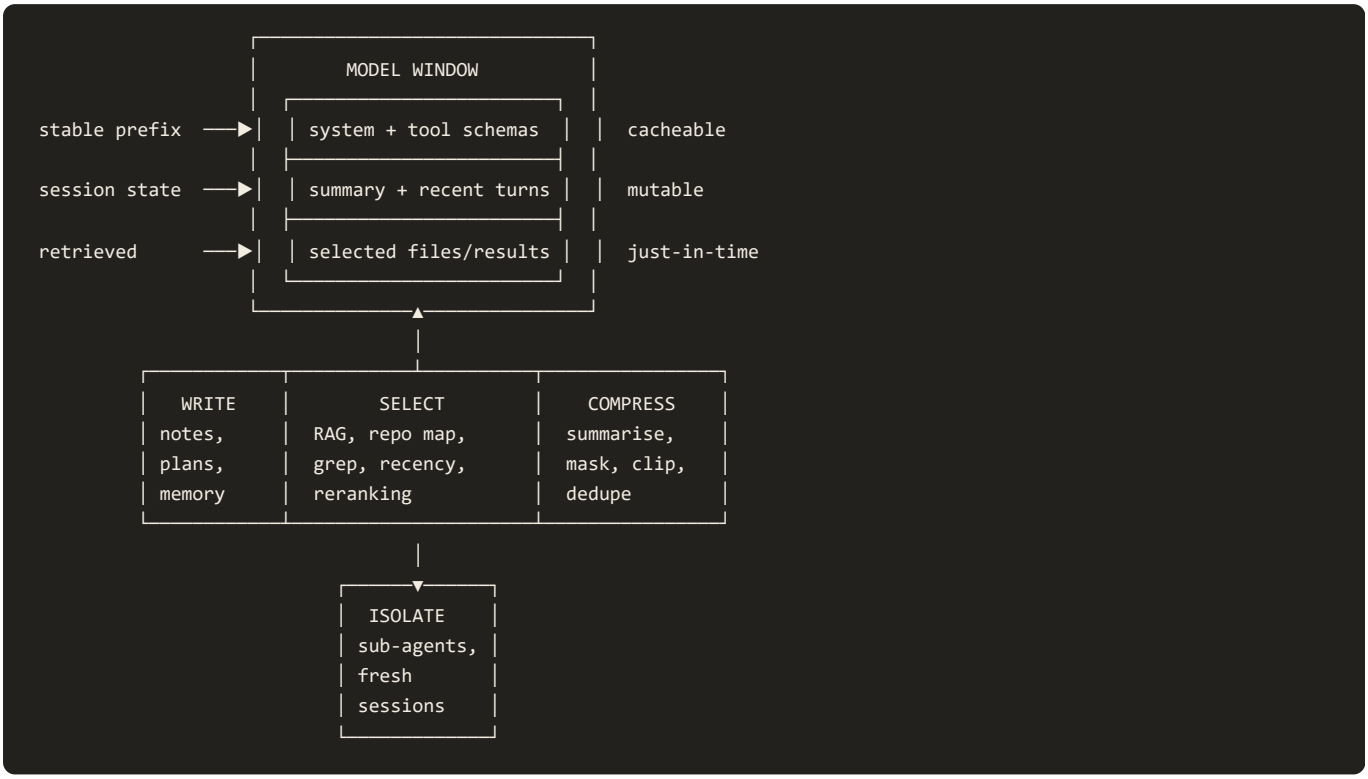
MISSING	CONSEQUENCE HERE	PRODUCTION ANSWER
Token counting	No idea whether the next request fits	Token meter in `PreModel`; provider count API
In-run compaction	A long run grows until the provider rejects it	Summarise old turns at a threshold
Observation masking	Old tool outputs are re-sent forever	Replace old results with placeholders
Structured summaries	Session trim loses the original goal	Pin a task/plan summary at the top
Retrieval over history	Old details are either present or gone	Search transcripts, embeddings, notes
Sub-agents	One window does all the work	Isolate research/exploration in child contexts
Prompt caching awareness	Every step re-pays for the stable prefix	Stable ordering + cache breakpoints
Cache-aware compaction	Not applicable yet	Compact at natural boundaries; re-warm cache

None of these is a bug in a teaching harness. Each one is a chapter-sized system, and the point of the app is that you can read the whole loop in one sitting. The next part is the map of those chapters.

Part III — What production systems do

7. The four moves

Every serious context-management system is a combination of four moves. I use the mnemonic **Write, Select, Compress, Isolate**.



MOVE	QUESTION IT ANSWERS	EXAMPLES
Write	"What should survive outside the window?"	Scratchpads, plans, `NOTES.md`, memory files, knowledge graphs, checkpointers
Select	"What should enter the window now?"	RAG, repo maps, grep, symbol search, recency/importance ranking
Compress	"How do I make what is already here smaller?"	Summarisation, compaction, observation masking, clipping, deduplication, reasoning stripping
Isolate	"What should not share a window at all?"	Sub-agents, plan/execute split, fresh sessions, sandboxed workers

Production harnesses do all four. This app does a little of **Write** (memory file), a little of **Select** (tools let the model choose what to read), a little of **Compress** (clipping, reasoning stripping) and none of **Isolate** (except that each task gets a fresh loop with a trimmed history).

8. Technique catalogue

8.1 Sliding window / FIFO truncation

What: Keep the most recent N messages or tokens; drop the oldest.

How: Count backwards until the budget is hit, then cut at a safe boundary. This app's `trimHistory` is a message-count sliding window with a `user`-boundary guard.

Trade-offs:

- ✅ trivial to implement, predictable, no extra model calls.
- ✅ good for chat-like interactions where recent turns matter most.
- ❌ loses the original goal and early decisions.
- ❌ message count is a poor proxy for token size.
- ❌ can break tool-call/tool-result pairing if the cut is naive.
- ❌ summarises nothing: the beginning is simply gone.

Fix for the goal-loss problem: pin the original task and a short plan into the system prompt or a "pinned facts" block that is never evicted. This is what structured note-taking does.

8.2 Structural truncation

What: Cut at boundaries that keep the conversation valid, not just at an arbitrary message index.

Why: Providers enforce role/sequence rules. A history that starts with a `tool` message, or ends with an assistant `tool_calls` that has no matching result, is rejected as malformed. If you clear tool outputs, you must clear the corresponding `tool_calls` too — or keep both.

How: When dropping old turns, drop whole **tool-call groups** (assistant `tool_calls` + all matching `tool` messages) rather than individual messages. When masking a tool result, keep the tool-call ID and replace only the content.

Trade-offs: More code and bookkeeping; the boundary search is cheap compared with the cost of a rejected request.

8.3 Summarisation / compaction

What: Replace a span of old messages with a shorter summary generated by a model (often a cheaper one).

How it usually works:

1. Trigger when input tokens cross a threshold (e.g. 70–90% of the window).
2. Keep the system prompt, the most recent N turns, and any pinned facts verbatim.
3. Summarise the middle: original task, what was tried, what worked, files changed, failing tests, open questions, constraints.
4. Replace the middle with one summary message.
5. Continue the loop; optionally re-read key files if the summary is not enough.

What a good summary preserves:

- the user's actual goal, in the user's words where possible,
- decisions and their reasons,
- exact identifiers: file paths, function names, commands, error strings,
- the current plan and what remains,
- failed approaches (so the agent does not repeat them).

What it should drop: raw file dumps, successful command output, repeated reads, reasoning traces, chit-chat.

Variants:

- **Rolling summary** — update one summary every N turns.
- **Hierarchical summary** — summaries of summaries, with the raw log kept on disk for audit.
- **Sectioned summary** — a structured template (`Goal / State / Files / Tests / Next`) rather than free prose.
- **Checkpoint compaction** — compact at task boundaries rather than mid-step.

Trade-offs:

- ✅ large token savings; can preserve semantic content that a positional cut would lose.
- ✅ reduces the "lost in the middle" problem by shortening the middle.
- ❌ lossy: exact strings, subtle constraints and provenance can vanish.
- ❌ summary of a summary drifts; errors compound.
- ❌ extra model call: cost, latency, and a new failure mode.
- ❌ invalidates prompt cache for everything after the summary point.
- ❌ hard to audit unless the raw transcript is retained.

Production examples: Claude Code exposes `/compact` and auto-compacts as the window fills; many chat products summarise old turns; agent frameworks offer "memory compaction" nodes. The design detail that matters is not the button — it is **what the summary is required to contain** and **whether the raw log is still available**.

8.4 Observation masking

What: Keep the conversation structure and the assistant's reasoning, but replace old **tool outputs** with a short placeholder:

```
[tool result omitted: 4,213 chars. Re-run read_file("src/agent.js") if needed.]
```

Why it is attractive: in agent runs, tool results are usually the largest and least reusable part of the context. The assistant's decisions and tool calls are small and carry the narrative. Masking preserves the narrative while deleting the bulk.

Evidence: a 2025 JetBrains Research paper, *The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management*, reported that simple masking matched or beat LLM summarisation on agent benchmarks at substantially lower cost. The lesson is not "never summarise"; it is "try the cheap structural fix before the expensive semantic one."

Trade-offs:

- ✅ no extra model call, deterministic, cache-friendly up to the masking point.
- ✅ preserves tool-call/result structure.
- ❌ the model may need a detail that was masked; it must re-read (extra step, latency).
- ❌ placeholders must include enough information to decide whether to re-fetch.
- ❌ does not help if the assistant's own messages are huge.

8.5 Context editing / clearing tool results

What: A provider-managed version of masking: the API removes old tool results when the context crosses a threshold, optionally keeping the N most recent tool uses and excluding specific tools.

Example: Anthropic's context-editing feature (beta, 2025) can clear tool uses/results automatically; the memory tool is often excluded so durable notes survive. This is attractive because it is declarative and happens inside the provider's request pipeline.

Trade-offs: less control than a custom policy; you must understand exactly what is removed; clearing a result without clearing its `tool_calls` can invalidate the request unless the provider handles the pairing for you.

8.6 Structured note-taking / scratchpads

What: The agent writes important state to a file — `NOTES.md`, `PLAN.md`, `progress.md`, a to-do list — and re-reads it after a compaction or at the start of the next session.

Why: Externalised state is not subject to the window. A summary can be regenerated from notes; a crash can be resumed; a human can inspect the plan.

What to write: goal, constraints, decisions, file map, commands that work, failing tests, open questions. **Not** a transcript and not "step 3 of 5" — task progress is exactly the kind of thing that goes stale.

Production examples: Claude Code's plan mode and `CLAUDE.md`-style project memory; the Anthropic memory tool's file-based memory directory; many agent frameworks' "scratchpad" or "state" objects.

Trade-offs:

- ✅ durable, auditable, human-editable, survives compaction and process restart.
- ✅ lets the agent "reload" its own context cheaply.
- ❌ the agent must remember to write and re-read; tool discipline is required.
- ❌ notes can go stale or contradict reality; they need timestamps and verification.
- ❌ a note-taking loop can itself become a context leak if every step writes a diary.

8.7 Tiered memory and paging

What: Treat memory as levels, not one array:

TIER	CONTENTS	LIFETIME
Working context	current messages, recent tool results	one request/step
Session state	summary, plan, pinned facts	one task/session
Episodic memory	past runs, transcripts, decisions	days/weeks
Semantic memory	facts, preferences, repo conventions	long-lived
Procedural memory	skills, workflows, prompts	long-lived

MemGPT / Letta popularised the OS analogy: a fixed main context plus external "recall" and "archival" storage, with the model calling functions to page data in and out. The model manages its own memory via tool calls.

Mem0, Zep/Graphiti and similar systems add extraction, deduplication, temporal validity and graph structure: facts have relationships and expiry; a new fact can invalidate an old one. Zep's temporal knowledge graph is designed to answer "what was true at time T?" rather than "what is the latest blob?"

Trade-offs:

- ✅ scales beyond any window; supports cross-session personalisation.
- ✅ graph/structured memory can resolve contradictions and time.
- ❌ extraction is lossy and can store wrong facts.
- ❌ retrieval quality becomes the bottleneck.
- ❌ more infrastructure, more failure modes, harder debugging.
- ❌ memory poisoning and privacy risks grow with persistence.

For a small harness, a markdown file with tags and dates (this app) is the honest 5% version of the same idea: human-readable, diffable, deletable, no service to run.

8.8 Retrieval-augmented generation (RAG)

What: Instead of putting a corpus in the prompt, store it in an index and retrieve the top-k chunks for the current query.

Pipeline: chunk → embed → index → query → hybrid search (dense + keyword) → rerank → pack into the prompt with provenance.

For code specifically:

- chunk by syntax (function, class, file), not fixed characters;
- store metadata: path, symbol, commit, language, last modified;
- combine embeddings with exact-match tools (`grep`, symbol search) because identifiers and error strings are poorly served by semantic similarity;
- version the index with the repo; stale indexes are worse than no index.

Trade-offs:

- ✅ sublinear context cost; can cover a huge corpus.
- ✅ fresh if the index is fresh.
- ❌ retrieval misses are silent and catastrophic: the model confidently answers without the needed chunk.
- ❌ chunking destroys cross-file context; reranking is another model and another cost.
- ❌ embeddings are bad at exact identifiers, numbers and code syntax.
- ❌ index freshness/staleness is an operational problem.

The retrieval-vs-long-context debate: stuffing a 200k-token repo into the window is sometimes better than a bad retriever, but it is slower, costlier and suffers context rot. The pragmatic answer is hybrid: retrieve candidates, then let the agent read the exact files it chooses.

8.9 Code-specific retrieval and repo maps

What: Build a compact structural map of the repository — files, symbols, definitions, references — and give the model that map instead of the files.

Aider's repo map is the canonical example: parse files with tree-sitter, build a graph of definitions and references, rank by relevance (including a PageRank-like importance score), and render the top symbols within a token budget. The agent sees "what exists and where", then reads only what it needs.

Other forms: ctags, LSP symbol indexes, call graphs, import graphs, Sourcegraph-style code search, GitHub/Copilot workspace indexing.

Trade-offs:

- ✅ far cheaper than file dumps; preserves structure and names.
- ✅ excellent for "where is X defined?" and change-impact questions.
- ❌ a map is not the code; the model still needs to read the target.
- ❌ parsers fail on exotic syntax; generated files and dynamic languages are harder.
- ❌ ranking can hide the one obscure file that matters.

8.10 Just-in-time context

What: Keep **identifiers** in context — file paths, URLs, search queries, symbol names — and load the underlying content only when the next step needs it.

This is the opposite of pre-loading everything. The agent sees a lightweight index or a list of candidate paths, then calls `read_file`, `grep`, a browser, or a database tool.

Why it works: the agent's own tool calls become the retrieval mechanism, guided by what it learns at each step. It avoids stale snapshots and lets the model adapt its search.

Trade-offs:

- ✅ freshest possible data; no index to maintain; naturally adaptive.
- ✅ context contains only what the agent actually used.
- ❌ each fetch costs a step and latency.
- ❌ the model must know what to look for; bad search queries waste steps.

- ❌ requires good tools and good tool descriptions.

This app is already just-in-time in spirit: `list_files` and `read_file` are the retrieval interface. The system prompt's "look before you write" rule is the policy that makes it work.

8.11 Tool-output management

The highest-leverage, least glamorous technique. Options, roughly in order of preference:

1. **Return a pointer, not the payload.** Write full output to a file, return the first/last N lines plus the path and a re-read instruction.
2. **Head + tail.** Keep the beginning (context) and the end (the error), drop the middle.
3. **Paginate.** `offset` / `limit` with an explicit "N more lines" marker (this app's `read_file`).
4. **Summarise at the tool boundary.** A test runner returns "42 passed, 3 failed: " instead of the full log.
5. **Diff instead of full file.** After an edit, return the hunk, not the file.
6. **Structured errors.** Return `exit code 1` plus the relevant stack frame, not 6,000 chars of build noise.
7. **Deduplicate.** If the same file was read twice, return a cache marker or a diff.
8. **Redact.** Secrets, tokens and personal data should never enter the transcript (this app's env allow-list is the same instinct).

Trade-offs: every transformation can hide the one line the model needed. The pointer pattern is the safest because the full data still exists and the model can ask for it. Head+tail is cheap but can cut the middle where the actual error is. Summaries require a model and can hallucinate.

8.12 Sub-agents and context isolation

What: A parent agent delegates a bounded task to a child with its own fresh context. The child explores, reads, searches, and returns a compact result (or writes a file the parent reads).

Why it helps: exploration is context-expensive and mostly disposable. A sub-agent can read 50 files, then return a 300-token answer. The parent's window stays clean.

Production examples: Claude Code sub-agents; orchestrator-worker patterns in multi-agent research systems; "research" agents that return citations; planner/executor splits.

Trade-offs:

- ✅ isolates noise; parallelisable; each child can use a different model/tool set.
- ✅ protects the main window from exploration exhaust.
- ❌ coordination overhead; duplicated work; children can miss the nuance the parent would have caught.
- ❌ the parent only sees the child's summary — a lossy channel.
- ❌ token usage can explode: Anthropic reported multi-agent systems using roughly 15× the tokens of a chat interaction, with token usage explaining most of the performance variance.
- ❌ harder observability: which child did what?

Sub-agents are the right tool when a task has a **narrow, well-specified sub-question** and the answer is much smaller than the search. They are the wrong tool for tightly coupled reasoning that needs the full conversation.

8.13 Prompt caching / KV cache

What: Providers can cache the computed key/value state for a stable prompt prefix, so repeated requests reuse it instead of re-processing the same tokens.

How to exploit it:

- Order context **stable** → **volatile**: system prompt, tool schemas, static docs, then session summary, then recent turns, then the new tool result.
- Do not put timestamps, request IDs or random ordering early.
- Use explicit cache breakpoints if the provider supports them (Anthropic `cache_control`; OpenAI does automatic prefix caching).
- Keep the prefix byte-identical across steps.
- Compact at natural boundaries and accept a cache miss; then re-warm.

Trade-offs:

- ✅ large cost and latency wins on long, repeated prefixes (often the system prompt + tools).
- ✅ can make a big context affordable.
- ❌ any change before the cache point invalidates everything after it.
- ❌ cache lifetime (TTL) may be shorter than your session.
- ❌ cached tokens are cheaper but not free; cache writes may cost more.
- ❌ **tension with compaction:** summarising the middle changes the prefix, so the next request is a cache miss. Sometimes it is cheaper to mask observations (which preserves the prefix up to the mask) than to rewrite history.

This tension is a good interview question: "*You can save 30k tokens by compacting, but it invalidates a 100k-token cached prefix. Do you compact?*" The answer depends on cache pricing, TTL, remaining steps and the quality cost of the larger context. Measure.

8.14 Token budgeting and measurement

What: Treat the window as a budget with line items, not a bucket.

A practical scheme:

LINE ITEM	BUDGET
System prompt + rules	1–2k
Tool schemas	1–3k
Pinned task/plan/facts	0.5–2k
Retrieved documents	20–40%
Recent conversation	20–30%

LINE ITEM	BUDGET
Tool observations	20–40%
Reserved for output	`max_tokens` + margin
Safety headroom	10–20%

Trigger policy (example, not gospel):

USAGE	ACTION
< 50%	Do nothing; monitor; cache
50–70%	Deduplicate, mask old observations, cap outputs
70–85%	Compact/summarise, retrieve instead of stuff, checkpoint notes
> 85%	Hard-evict oldest raw turns, spawn a sub-agent, or stop and ask

Measure:

- input/output tokens per step and per run,
- cache hit rate,
- cost per task and per successful task,
- steps per task,
- largest messages and which tool produced them,
- compaction count and post-compaction task success.

Pitfalls:

- counting characters and calling it tokens,
- forgetting tool schemas and chat-template overhead,
- budgeting to 100% of the window,
- assuming a provider's advertised window is the effective window,
- not accounting for the output reservation,
- different tokenizers for different models.

8.15 Compression research: token-level pruning

What: Instead of summarising sentences, remove or merge low-information tokens. **LLMLingua** and **LLMLingua-2** use a small language model to estimate token importance and compress prompts; **Selective Context** drops redundant content; **gist tokens** compress instructions into learned tokens.

Trade-offs:

- ✅ can achieve large compression ratios with modest quality loss on some tasks.
- ✅ cheap at inference time for long documents.
- ❌ lossy in ways that are hard to predict; exact identifiers and negation can break.
- ❌ requires a compressor model and tuning; distribution mismatch with the target model.
- ❌ mostly used for retrieval contexts and long documents, not for live agent state.

For agents, **structural** compression (masking, pointers, dedupe) is usually safer than **token-level** compression, because it preserves the conversation's logic.

8.16 Session persistence and checkpointing

What: Save the conversation/state so a run can resume after a crash, a restart, or a human pause.

Production examples: LangGraph checkpointers (thread state persisted at each node), OpenAI Agents SDK sessions (conversation history stored and rehydrated), durable-execution engines (Temporal-style workflows), and simple JSONL transcripts.

Why it is context management: persistence lets you **discard** working context safely. If the full transcript is on disk, the window can hold a summary plus a pointer; if the process dies, the state is not lost.

Trade-offs:

- ✅ resumability, auditability, human handoff.
- ✅ decouples "what happened" from "what is in the window".
- ❌ storage, schema migration, privacy/retention.
- ❌ a resumed agent may act on stale state; version the checkpoint.
- ❌ transcripts contain secrets unless redacted.

8.17 Bigger windows and context rot

What: Use a model with a 200k, 1M or 2M token window and put more in.

Why it is not a strategy by itself:

- **Lost in the middle:** retrieval accuracy is U-shaped; the middle of a long context is used less reliably (Liu et al., 2023).
- **Context rot:** performance degrades as input length grows even on simple tasks; needle-in-a-haystack scores overstate real ability (Chroma, 2025).
- **RULER:** many models' *effective* context is much shorter than their advertised maximum (Hsieh et al., 2024).
- **Cost and latency:** prefill scales with length; long prompts are slow and expensive.
- **Distraction:** irrelevant context can actively hurt, not just fail to help.

The right use of a big window: short bursts of high-relevance material (a whole module, a large diff, a long transcript for a one-off analysis), combined with retrieval and compaction. Not "dump the repo and hope".

8.18 Security and trust in the context

Anything that enters the context can influence the model. That makes context a security boundary:

- **Prompt injection** via retrieved web pages, issues, READMEs, tool output, memory files.
- **Memory poisoning** across sessions: an attacker gets a false "fact" into long-term memory.
- **Secret leakage** into transcripts, summaries and logs.
- **Provenance loss:** after summarisation, the model may not know which claim came from an untrusted source.

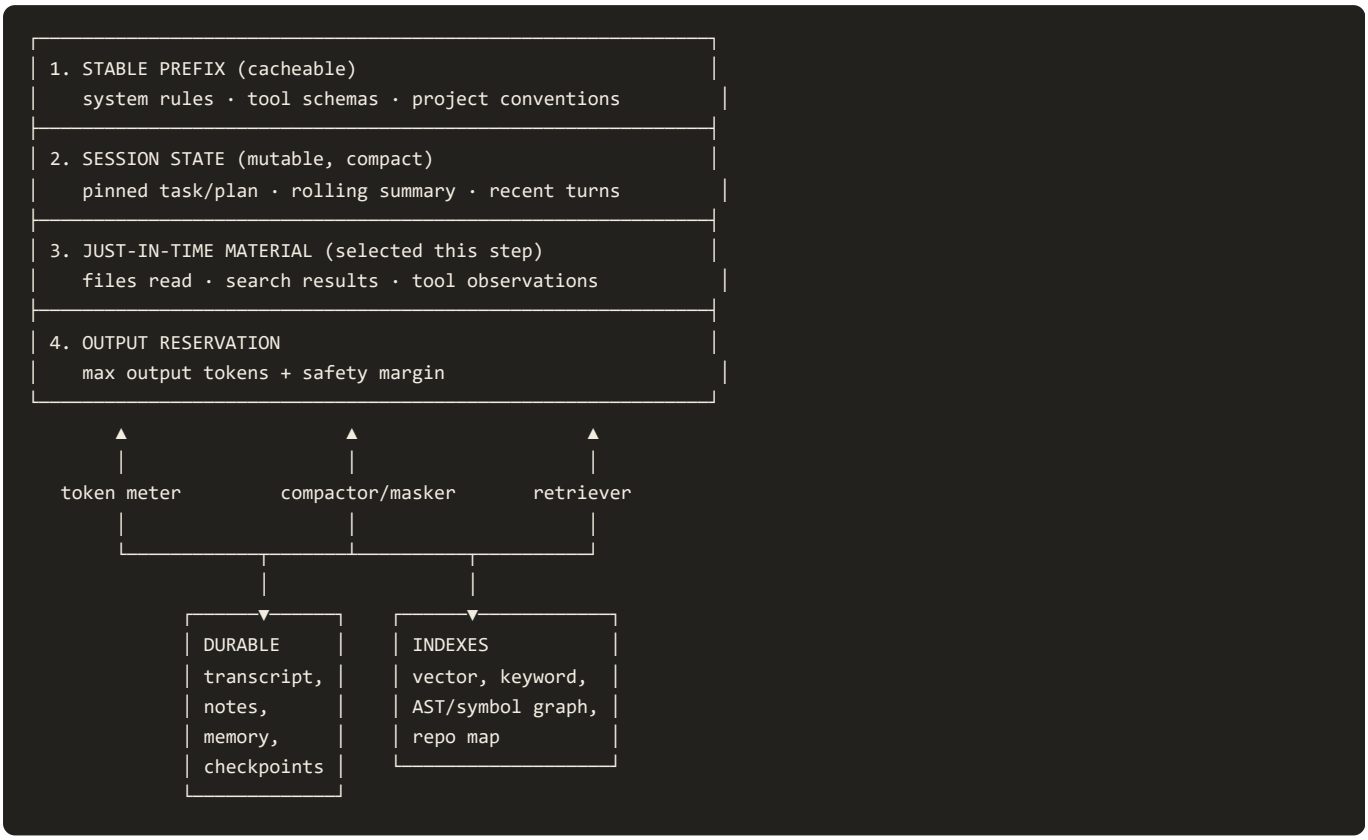
Mitigations:

- label content with provenance and trust level (`untrusted web page`, `repo file`, `user instruction`);
- never let retrieved content override system rules silently;
- keep secrets out of the context (allow-list env vars, redact in hooks);
- review memory writes, add timestamps and TTLs;
- sandbox tool execution (this app's write fence and command policy);
- treat summaries as derived, not authoritative; keep the raw source for audit.

---

9. Putting it together: a reference architecture

A production context pipeline often looks like this:



The loop repeats: measure → select → compress → call → observe → write.

A concrete step:

1. `PreModel` counts tokens. If over 70%, run the compactor.
2. Compactor keeps the system prompt, pinned facts and last 6 turns; summarises the middle into a structured block; masks old tool outputs older than 2 steps.
3. Retriever adds only the files the current plan names.
4. Model call with a stable prefix and a reserved output budget.
5. `PostToolUse` caps a 40k-char test log to head+tail + a pointer to the full file.
6. `Stop` / `SessionEnd` writes the transcript and updates the notes/memory.

That is the same loop as this app, with four more services attached at the seams.

---

10. Complexity and trade-offs

No technique dominates. The matrix below is qualitative; the point is to make the trade-off explicit before you build.

TECHNIQUE	TOKEN SAVINGS	FIDELITY RISK	EXTRA LATENCY	BUILD EFFORT	CACHE-FRIENDLY	BEST FOR
Sliding window	Medium	High (loses old goal)	None	Low	✅	Chat, short sessions
Structural cut	Medium	Medium	None	Low	✅	Tool-heavy histories
Observation masking	High	Medium (must re-read)	None	Low	Mostly ✅	Agent tool output
Pointer outputs	High	Low (data still exists)	One re-read	Low–Med	✅	Huge logs/files
Clipping/pagination	High	Low–Med	None	Low	✅	Tool design
Summarisation	High	High (lossy)	One model call	Medium	❌ (invalidates)	Long sessions
Hierarchical summary	Very high	Very high (drift)	Multiple calls	High	❌	Very long sessions
Structured notes	Medium	Low (human-auditable)	Write/read calls	Low–Med	✅	Multi-task work
RAG	Very high	High (retrieval miss)	Query + rerank	High	Partial	Large corpora
Repo map	High	Medium	Parse/index	Medium–High	✅	Code navigation
Just-in-time reads	High	Low (fresh)	One step per fetch	Low	✅	Code agents
Sub-agents	Very high (main window)	Medium (summary channel)	Coordination	High	✅	Exploration
Prompt caching	Cost/latency, not capacity	None	None	Low–Med	✅	Stable prefixes
Compaction + cache	High	High	Cache miss	High	❌	Long runs
Bigger window	Negative (uses more)	Quality risk (rot)	Higher prefill	None	Partial	One-off large inputs

The tensions worth teaching

1. **Fidelity vs budget.** Every compression is a bet that the dropped detail will not matter. The bet is sometimes wrong; design for recovery (pointers, re-read tools).
2. **Cache vs mutation.** Compaction saves tokens but invalidates the prefix. Masking and pointer outputs preserve more of the prefix. The cheapest token is a cached one; the second cheapest is one you never generated.
3. **Retrieval vs reasoning.** Retrieval adds a step and can miss. Stuffing adds tokens and can distract. The best systems let the model choose when to retrieve, with good tools and good descriptions.
4. **Isolation vs coherence.** Sub-agents protect the main window but lose nuance. Keep tightly coupled reasoning in one window; isolate exploration.

5. **Automation vs control.** Auto-compaction that surprises the user is worse than a visible `/compact`. Make context changes observable.
6. **Capacity vs quality.** The effective window is smaller than the advertised one. Budget for quality, not just fit.
7. **Proactive vs reactive.** Reactive compaction at 95% is cheaper on average but risks a failed request. Proactive compaction at 70% costs more but is predictable. Many systems use both: soft threshold + hard fallback.
8. **Cost vs latency.** Caching reduces both, but only for stable prefixes. Bigger models with bigger windows may reduce steps but increase per-token cost.
9. **Observability vs privacy.** Transcripts are essential for debugging and dangerous for secrets. Redact at the boundary.
10. **Summary drift.** A summary of a summary of a summary is a rumour. Pin facts verbatim; keep the raw log; re-verify when it matters.

A practical decision rule

> **Prefer the cheapest transformation that preserves the ability to recover the original.**
> Pointer > mask > clip > summarise > drop.
> A pointer keeps the data. A mask keeps the structure. A clip keeps the edges. A summary keeps the gist. A drop keeps nothing.

Then measure whether the cheap transformation was enough. Most agents do not need a summariser; they need better tool outputs and a pinned plan.

---

11. Failure modes checklist

SYMPTOM	LIKELY CAUSE	FIX
`context_length_exceeded` / 400 near the end of a run	No token meter; one huge tool result	Budget in `PreModel`; cap/pointer large outputs
Agent forgets the original task after a trim	Positional cut dropped the goal	Pin task + plan; structured note
Agent re-reads the same file every step	Old observation was evicted	Keep recent reads; cache file results; pointer
Agent repeats a failed approach	Failure was summarised away	Preserve failed attempts and error strings in the summary
Compaction loses exact paths/symbols	Free-prose summary	Structured summary; keep identifiers verbatim
Request rejected after clearing tool output	`tool_calls` / `tool` pairing broken	Clear whole tool groups; structural cut
Cost spikes with steps	Quadratic re-send of history	Cache, mask, compact, fewer steps
Quality drops at 30% of window	Context rot / distraction	Retrieve less; curate; evaluate effective context
Summary contains invented facts	Summariser hallucinated	Summarise from raw text with citations; verify
Cache hit rate near zero	Unstable prefix (timestamps, reordered tools)	Stable ordering; cache breakpoints
Memory confidently wrong	Poisoned or stale memory	Provenance, timestamps, TTL, review, `forget`
Secrets in transcript	Tool output or env inherited	Redact in `PostToolUse`; env allow-list
Agent loops forever	No step budget or loop detector	`maxSteps`; detect repeated calls/hashes

12. How to evaluate context management

Context management is an empirical discipline. Define metrics before changing anything:

Efficiency

- input tokens per step and per task,
- output tokens per task,
- cache hit rate,
- cost per task and per **successful** task,
- steps per task,
- tokens per successful tool call.

Quality

- task success rate (tests pass, answer correct),
- number of retries / repeated actions,
- constraint violations (edited the wrong file, ignored a rule),
- post-compaction success (does the agent still know the task?),
- retrieval precision/recall on a labelled set.

Robustness

- failures at 50%, 70%, 90% window usage,
- behaviour after a cache miss,
- behaviour when a tool returns 10× the expected output,
- resume-from-checkpoint correctness.

A useful regression test: after every compaction, ask the agent a "canary" question whose answer was in the dropped span — the original task, a path, a failing test name. If it cannot answer and cannot re-derive it, the summary is not good enough.

Benchmarks to know: needle-in-a-haystack (necessary but not sufficient), multi-needle, LongBench, RULER, and agent-specific benchmarks (SWE-bench, τ-bench) with token/cost reporting. Always report tokens alongside success; a 2% gain for 5× the tokens is usually a loss.

---

13. Exercises for this repository

These are ordered from "one hook" to "a real feature". Each one teaches a different part of the trade-off.

1. **Add a token meter.** In `PreModel`, estimate `chars / 4` per message, print the total and the top five largest messages with their roles/tools. Run the demo and watch the array grow. *Teaches: where the tokens actually are.*
2. **Turn a huge observation into a pointer.** In `PostToolUse`, if a result exceeds ~4,000 chars, write the full text to `workspace/.agent/observations/<tool>-<step>.txt` and return the first/last 20 lines plus the path. *Teaches: recoverable compression.*
3. **Implement observation masking.** Add a small helper in `agent.js` (or extend the hook contract so `PreModel` can return `{ messages }`) that replaces tool results older than 2 steps with `[masked: N chars — re-read <path> if needed]`. Keep the `tool_call_id`. *Teaches: structure-preserving compression and the difference between masking and summarising.*
4. **Compact at 70%.** Write a `PreModel` hook that, when the estimate crosses a threshold, calls a cheap model with the middle turns and a structured summary prompt (`Goal / State / Files / Tests / Next`), then replaces those turns. Keep the system prompt, the last N turns and pinned facts. *Teaches: lossy compression, cache invalidation and summary design.*
5. **Turn memory injection off and measure.** Run the same multi-task session with `MEMORY_PROMPT_CHARS=1200` and `MEMORY_PROMPT_CHARS=0` (with `recall` still available). Compare steps, tokens and answer quality. *Teaches: eager vs lazy retrieval.*
6. **Stress `read_file`.** Generate a 2,000-line file with long lines, read it at `limit: 2000`, and measure the context cost. Then design a byte/token cap and a pointer response. *Teaches: why line caps are not token caps.*
7. **Build a repo-map tool.** Parse the workspace with a simple regex or tree-sitter, rank symbols by references, and return the top N within a character budget. Compare navigation with and without it. *Teaches: structural retrieval and token-budgeted selection.*
8. **Spawn a sub-agent.** Add a tool that calls `runTask` with a narrow research task and a fresh history, returning only the final answer. Give it a small `maxSteps`. *Teaches: context isolation and the summary channel.*

---

14. Glossary

TERM	MEANING
Context window	Maximum tokens a model can process in one request (input + output).
Token	A subword unit of text; roughly 4 characters of English, less for code.
Prefill	The provider's processing of the input prompt before generating output.
KV cache	Stored attention key/value state that lets a provider skip recomputing a repeated prefix.
Prompt caching	Provider feature that reuses the KV cache for a stable prompt prefix, at lower cost/latency.
Cache breakpoint	An explicit marker telling a provider where a cacheable prefix ends.
Compaction	Replacing old context with a shorter summary.
Observation masking	Replacing old tool outputs with placeholders while keeping the conversation structure.
Context editing	Provider-managed removal of old tool results/content when a threshold is crossed.
RAG	Retrieval-augmented generation: retrieve relevant chunks, then put them in the prompt.
Reranker	A model that reorders retrieved candidates by relevance before packing.
Repo map	A compact structural summary of a codebase (files, symbols, references) within a token budget.
Sub-agent	A child agent with its own context window that returns a condensed result to a parent.
Scratchpad / notes	A durable file where the agent externalises plan and state.
Checkpoint	A persisted snapshot of agent/session state, used for resume and audit.
Working set	The subset of all available information currently in the window.
Eviction	Removing items from the window by a policy (oldest, least relevant, lowest importance).
Context rot	Degradation in model performance as input length grows, even when the needed information is present.
Lost in the middle	The tendency to use information at the start/end of a long context more reliably than the middle.
Effective context	The length at which a model still performs well, as opposed to its advertised maximum.
Token budget	An allocation of the window across system, tools, retrieval, history, observations and output.
Semantic / episodic memory	Long-term memory for facts/concepts vs specific past events/transcripts.
Memory poisoning	Introducing false or malicious content into long-term memory so future runs trust it.

15. Further reading

SOURCE	WHY IT IS WORTH READING
Anthropic, *Effective context engineering for AI agents* (2025) — anthropic.com/engineering	The "smallest set of high-signal tokens" framing; compaction, structured note-taking, sub-agents, just-in-time retrieval.
Anthropic, *Building effective agents* (2024) — anthropic.com/engineering	Workflows vs agents; orchestrator-worker; when not to add complexity.
Liu et al., *Lost in the Middle: How Language Models Use Long Contexts* (2023) — arXiv:2307.03172	The U-shaped attention result; why position matters.
Hsieh et al., *RULER: What's the Real Context Size of Your Long-Context Language Models?* (2024) — arXiv:2404.06654	Effective context is much shorter than advertised.
Chroma, *Context Rot: How Increasing Input Tokens Impacts LLM Performance* (2025) — research.trychroma.com	Length alone degrades performance; needle tests are not enough.
Packer et al., *MemGPT: Towards LLMs as Operating Systems* (2023) — arXiv:2310.08560	Tiered memory and model-driven paging.
Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (2020) — arXiv:2005.11401	The original RAG formulation.
Jiang et al., *LLMLingua* (2023) — arXiv:2310.05736; *LLMLingua-2* (2024) — arXiv:2403.12968	Token-level prompt compression and its trade-offs.
JetBrains Research, *The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management* (2025)	Strong evidence for trying masking before summarisation.
Zep / Graphiti — arXiv:2501.13956 (Zep paper); Mem0 — arXiv:2504.19413	Temporal knowledge graphs and production memory extraction.
Aider repo map — aider.chat/docs/repomap.html	Token-budgeted structural retrieval for code.
Claude Code docs — docs.anthropic.com/en/docs/claude-code	Compaction, sub-agents, project memory and plan mode in a shipping tool.
Anthropic context editing / memory tool docs (beta, 2025)	Provider-managed tool-result clearing and file-based memory.
OpenAI Agents SDK sessions; LangGraph persistence docs	Session history and checkpointing patterns.
LongBench — arXiv:2308.14508; SWE-bench; τ-bench	Evaluation beyond needle-in-a-haystack.

16. The one thing to remember

The context window is not storage. It is the model's **working memory for exactly one thought**.

This app's answer is honest and small: bound the steps, bound each observation, bound the memory injection, strip what the model does not need, and cut history at a safe boundary. A production answer keeps the same shape but adds measurement, compaction, retrieval, isolation and caching — and then spends most of its engineering effort on the hard part: **deciding what is worth remembering, and proving that dropping the rest did not break the task.**